

Operating System

Round Robin vs Priority Comparison Project

David Elks Fam – Mostafa Sobhy – Marwan Ragab

Asmaa Hesham – Menna Mohammed – Shama Ahmed

Test Scenarios

Scenario A: General Case

Process ID	Arrival Time	Burst Time	Priority
P1	0	4	2
P2	1	3	1
P3	2	2	3

Time Quantum = 2, Lower number has higher priority

Using Round Robin:

We give equal splits of 2sec to each process, then it returns back to the ready queue, as shown in the below Gantt chart.

P1	P2	P3	P1	P2
0 - 2	2 - 4	4 - 6	6 - 8	8 - 9

using the time formulas below

- *Turnaround Time (TAT) = Completion Time - Arrival Time*
- *Waiting Time (WT) = Turnaround Time - Burst Time*
- *Response Time (RT) = Time of First CPU Allocation - Arrival Time*
- *Completion Time (CT) = The exact clock time when a process fully terminates*

Time Analysis of the scenario is:

Process	Completion Time	Turnaround Time (TAT)(CT - Arrival)	Waiting Time (WT)(TAT - Burst)	Response Time (RT)(First CPU Time - Arrival)
P1	8	$8 - 0 = 8$	$8 - 4 = 4$	$0 - 0 = 0$
P2	9	$9 - 1 = 8$	$8 - 3 = 5$	$2 - 1 = 1$
P3	6	$6 - 2 = 4$	$4 - 2 = 2$	$4 - 2 = 2$

- Average Turnaround Time: $(8 + 8 + 4) / 3 = 6.67$ units
- Average Waiting Time: $(4 + 5 + 2) / 3 = 3.67$ units
- Average Response Time: $(0 + 1 + 2) / 3 = 3 / 3 = 1.0$ unit

Using Priority Preemptive:

The CPU is always given to the process with the highest priority (the lowest number). If a new process arrives with a higher priority than the one currently running, the running process is immediately interrupted (preempted).

P1	P2	P1	P3
0 - 1	1 - 4	4 - 7	7 - 9

Using the above Time formulas:

Process	Completion Time	Turnaround Time (TAT)(CT - Arrival)	Waiting Time (WT)(TAT - Burst)	Response Time (RT)(First CPU Time - Arrival)
P1	7	$7 - 0 = 7$	$7 - 4 = 3$	$0 - 0 = 0$
P2	4	$4 - 1 = 3$	$3 - 3 = 0$	$1 - 1 = 0$
P3	9	$9 - 2 = 7$	$7 - 2 = 5$	$7 - 2 = 5$

- Average Turnaround Time: $(7 + 3 + 7) / 3 = 5.67$ units
- Average Waiting Time: $(3 + 0 + 5) / 3 = 2.67$ units
- Average Response Time: $(0 + 0 + 5) / 3 = 5 / 3 = 1.67$ units

Simulator Results

1. Round Robin Scheduling

GANTT CHART



PID	Arrival	Burst	Priority	CT	TAT	WT	RT
p1	0	4	2	8	8	4	0
p2	1	3	1	9	8	5	1
p3	2	2	3	6	4	2	2

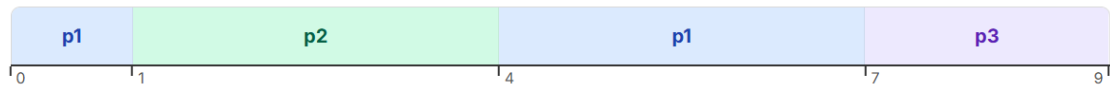
Avg TAT: **6.67**

Avg WT: **3.67**

Avg RT: **1**

2. Preemptive Priority Scheduling

GANTT CHART



PID	Arrival	Burst	Priority	CT	TAT	WT	RT
p1	0	4	2	7	7	3	0
p2	1	3	1	4	3	0	0
p3	2	2	3	9	7	5	5

Avg TAT: **5.67**

Avg WT: **2.67**

Avg RT: **1.67**

Analysis Questions

1. Which algorithm gave better average waiting time?

Preemptive Priority.

2. Which algorithm gave better average response time?

Round Robin.

3. Did higher-priority processes gain significant advantage?

Yes. The most urgent process (P2) finished at time 4 in Priority, vs 9 in RR.

4. Did Round Robin appear more balanced across all processes?

Yes. Maximum wait time in RR was 5 ms, compared to 5 ms in Priority.

5. Was starvation observed or likely in Priority Scheduling?

Yes, there is evidence of starvation risk from how P3 was treated, but starvation itself didn't occur.

6. Which algorithm would you recommend?

Round Robin is highly recommended for this workload to guarantee fairness and prevent starvation.

Conclusion

- **Overall Performance:** On the selected datasets, Preemptive Priority performed better overall based on standard average metrics.
- **Urgent-Task Treatment:** Priority-based service successfully improved urgent-task treatment in this scenario.
- **Fairness Improvement:** Round Robin improved fairness by restricting the maximum wait time any single process had to endure.
- **Starvation Risk:** Starvation risk appeared during the Priority scheduling simulation.

Scenario B: Urgency

A heavy background process is interrupted by two highly urgent, short-burst process

Process ID	Arrival Time	Burst Time	Priority
Heavy	0	10	5
Urgent1	2	2	1
Urgent2	3	2	2

Using Round Robin: We give equal splits of 2sec to each process, then it returns back to the ready queue, as shown in the below Gantt chart.

Heavy	Urgent1	Heavy	Urgent2	Heavy	Heavy	Heavy
0 - 2	2 - 4	4 - 6	6 - 8	8 - 10	10 - 12	12 - 14

Process	Completion Time	Turnaround Time (TAT)(CT - Arrival)	Waiting Time (WT)(TAT - Burst)	Response Time (RT)(First CPU Time - Arrival)
Heavy	14	$14 - 0 = 14$	$14 - 10 = 4$	$0 - 0 = 0$
Urgent1	4	$4 - 2 = 2$	$2 - 2 = 0$	$2 - 2 = 0$
Urgent2	8	$8 - 3 = 5$	$5 - 2 = 3$	$6 - 3 = 3$

- Average Turnaround Time: $(14 + 2 + 5) / 3 = 21 / 3 = 7.0$ units
- Average Waiting Time: $(4 + 0 + 3) / 3 = 7 / 3 = 2.33$ units
- Average Response Time: $(0 + 0 + 3) / 3 = 3 / 3 = 1.0$ unit

Using Priority Preemptive: The CPU is always given to the process with the highest priority (the lowest number. Heavy process is interrupted by the urgent processes

Heavy	Urgent1	Urgent2	Heavy
0 - 2	2 - 4	4 - 6	6 - 14

Process	Completion Time	Turnaround Time (TAT)(CT - Arrival)	Waiting Time (WT)(TAT - Burst)	Response Time (RT)(First CPU Time - Arrival)
Heavy	14	$14 - 0 = 14$	$14 - 10 = 4$	$0 - 0 = 0$
Urgent1	4	$4 - 2 = 2$	$2 - 2 = 0$	$2 - 2 = 0$
Urgent2	6	$6 - 3 = 3$	$3 - 2 = 1$	$4 - 3 = 1$

- Average Turnaround Time: $(14 + 2 + 3) / 3 = 19 / 3 = 6.33$ units
- Average Waiting Time: $(4 + 0 + 1) / 3 = 5 / 3 = 1.67$ units
- Average Response Time: $(0 + 0 + 1) / 3 = 1 / 3 = 0.33$ units

Simulator Results

1. Round Robin Scheduling

GANTT CHART



PID	Arrival	Burst	Priority	CT	TAT	WT	RT
Heavy	0	10	5	14	14	4	0
Urgent1	2	2	1	4	2	0	0
Urgent2	3	2	2	8	5	3	3

Avg TAT: 7

Avg WT: 2.33

Avg RT: 1

2. Preemptive Priority Scheduling

GANTT CHART



PID	Arrival	Burst	Priority	CT	TAT	WT	RT
Heavy	0	10	5	14	14	4	0
Urgent1	2	2	1	4	2	0	0
Urgent2	3	2	2	6	3	1	1

Avg TAT: 6.33

Avg WT: 1.67

Avg RT: 0.33

Analysis Questions

1. Which algorithm gave better average waiting time?

Preemptive Priority gave a significantly better average waiting time (1.67 units compared to 2.33 units in Round Robin).

2. Which algorithm gave better average response time?

Preemptive Priority (0.33 units compared to 1.0 unit in Round Robin).

3. Did higher-priority processes gain significant advantage?

Yes. Priority scheduling allowed *both* urgent tasks to finish back-to-back by time 6. In Round Robin, Urgent2 was forced to wait behind another 2-second slice of the Heavy task, delaying its completion until time 8.

4. Did Round Robin appear more balanced across all processes?

Round Robin forced the CPU to go back to the Heavy process before finishing Urgent2. While this is "fairer" to the Heavy process, it degraded the overall system performance averages.

5. Was starvation observed or likely in Priority Scheduling?

Starvation didn't occur because there were only two urgent tasks, but the Heavy process was continuously pushed back. If more urgent tasks kept arriving, the Heavy task would certainly face starvation.

6. Which algorithm would you recommend?

Preemptive Priority is highly recommended for this scenario.

Conclusion:

Overall Performance: On this dataset, Preemptive Priority performed significantly better across all average metrics (TAT, WT, and RT).

- **Urgent-Task Treatment:** Priority scheduling successfully grouped and executed the urgent tasks immediately, preventing them from being delayed by the Heavy background task.
- **Fairness Improvement:** Round Robin provided fair time-slicing by giving the Heavy task CPU time in the middle of the urgent tasks, but this came at the cost of overall system efficiency.
- **Starvation Risk:** The Heavy task was delayed significantly, highlighting the risk of starvation if a steady stream of high-priority processes were to enter the system.

Scenario C: Fairness Case

In this case all the 3 processes have the same priority and same arrival time, but different bursts times to test which algorithm perform the best on that case.

Process ID	Arrival Time	Burst Time	Priority
P1	0	10	2
P2	0	2	2
P3	0	2	2

Using Round Robin:

We give equal splits of 2sec to each process, then it returns back to the ready queue, as shown in the below Gantt chart.

P1	P2	P3	P1
0 - 2	2 - 4	4 - 6	6 - 14

Using Time Formulas:

Process	Completion Time	Turnaround Time (TAT)(CT - Arrival)	Waiting Time (WT)(TAT - Burst)	Response Time (RT)(First CPU Time - Arrival)
P1	14	$14 - 0 = 14$	$14 - 10 = 4$	$0 - 0 = 0$
P2	4	$4 - 0 = 4$	$4 - 2 = 2$	$2 - 0 = 2$
P3	6	$6 - 0 = 6$	$6 - 2 = 4$	$4 - 0 = 4$

- Average Turnaround Time: $(14 + 4 + 6) / 3 = 24 / 3 = 8.0$ units
- Average Waiting Time: $(4 + 2 + 4) / 3 = 10 / 3 = 3.33$ units
- Average Response Time: $(0 + 2 + 4) / 3 = 6 / 3 = 2.0$ units

Using Priority Preemptive:

Since they are all having the same priority, the priority preemptive treat them equally and run them in sequential order.

P1	P2	P3
0 - 10	10 - 12	12 - 14

Using Time Formulas

Process	Completion Time	Turnaround Time (TAT)(CT - Arrival)	Waiting Time (WT)(TAT - Burst)	Response Time (RT)(First CPU Time - Arrival)
P1	10	$10 - 0 = 10$	$10 - 10 = 0$	$0 - 0 = 0$
P2	12	$12 - 0 = 12$	$12 - 2 = 10$	$10 - 0 = 10$
P3	14	$14 - 0 = 14$	$14 - 2 = 12$	$12 - 0 = 12$

- Average Turnaround Time: $(10 + 12 + 14) / 3 = 36 / 3 = 12.0$ units
- Average Waiting Time: $(0 + 10 + 12) / 3 = 22 / 3 = 7.33$ units
- Average Response Time: $(0 + 10 + 12) / 3 = 22 / 3 = 7.33$ units

Simulator Results

Simulation Results

1. Round Robin Scheduling

GANTT CHART



PID	Arrival	Burst	Priority	CT	TAT	WT	RT
P1	0	10	2	14	14	4	0
P2	0	2	2	4	4	2	2
P3	0	2	2	6	6	4	4

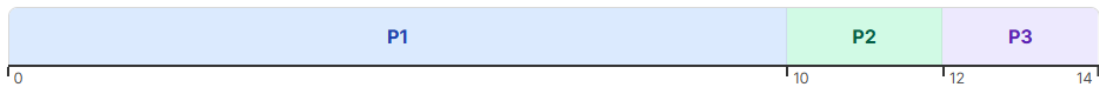
Avg TAT: 8

Avg WT: 3.33

Avg RT: 2

2. Preemptive Priority Scheduling

GANTT CHART



PID	Arrival	Burst	Priority	CT	TAT	WT	RT
P1	0	10	2	10	10	0	0
P2	0	2	2	12	12	10	10
P3	0	2	2	14	14	12	12

Avg TAT: 12

Avg WT: 7.33

Avg RT: 7.33

Analysis Questions:

1. Which algorithm gave better average waiting time?

Round Robin (3.33 units compared to 7.33 units in Priority).

2. Which algorithm gave better average response time?

Round Robin (2.0 units compared to 7.33 units in Priority).

3. Did higher-priority processes gain significant advantage?

No. Because all processes had the exact same priority (2), the Priority scheduling algorithm degraded into standard First-Come-First-Serve based on Process ID, providing no priority-based advantages.

4. Did Round Robin appear more balanced across all processes?

Yes. Round Robin allowed the short processes (P2 and P3) to finish much earlier, breaking up the long CPU burst of P1.

5. Was starvation observed or likely in Priority Scheduling?

Starvation wasn't present (as no new higher-priority processes arrived to interrupt)

6. Which algorithm would you recommend?

Round Robin is highly recommended for this workload. It drastically improved both average waiting time and average response time.

Conclusion:

- **Overall Performance:** On the selected dataset, Round Robin performed significantly better across all average metrics.
- **Urgent-Task Treatment:** Because all tasks shared the same priority level, the Priority algorithm failed to prioritize shorter tasks, resulting in poor performance.
- **Fairness Improvement:** Round Robin provided a massive improvement in fairness, giving P2 and P3 immediate attention rather than making them wait for P1's massive 10-unit burst to finish.
- **Starvation Risk:** Didn't occur since they all have the same priority.

Scenario D (Starvation)

Multiple high priority processes dominate execution

Process ID	Arrival Time	Burst Time	Priority
LowPri	0	2	10
High1	1	4	1
High2	2	4	1
High3	3	4	1

Using Round Robin:

We give equal splits of 2sec to each process, then it returns back to the ready queue, as shown in the below Gantt chart.

LowPri	High1	High2	High3	High1	High2	High3
0 - 2	2 - 4	4 - 6	6 - 8	8 - 10	10 - 12	12 - 14

Process	Completion Time	Turnaround Time (TAT)(CT - Arrival)	Waiting Time (WT)(TAT - Burst)	Response Time (RT)(First CPU Time - Arrival)
LowPri	2	$2 - 0 = 2$	$2 - 2 = 0$	$0 - 0 = 0$
High1	10	$10 - 1 = 9$	$9 - 4 = 5$	$2 - 1 = 1$
High2	12	$12 - 2 = 10$	$10 - 4 = 6$	$4 - 2 = 2$
High3	14	$14 - 3 = 11$	$11 - 4 = 7$	$6 - 3 = 3$

- Average Turnaround Time: $(2 + 9 + 10 + 11) / 4 = 32 / 4 = 8.0$ units
- Average Waiting Time: $(0 + 5 + 6 + 7) / 4 = 18 / 4 = 4.5$ units
- Average Response Time: $(0 + 1 + 2 + 3) / 4 = 6 / 4 = 1.5$ units

Using Priority Preemptive:

The CPU is always given to the process with the highest priority (the lowest number). If a new process arrives with a higher priority than the one currently running, the running process is immediately interrupted (preempted).

LowPri	High1	High2	High3	LowPri
0 - 1	1 - 5	5 - 9	9 - 13	13 - 14

Process	Completion Time	Turnaround Time (TAT)(CT - Arrival)	Waiting Time (WT)(TAT - Burst)	Response Time (RT)(First CPU Time - Arrival)
LowPri	14	$14 - 0 = 14$	$14 - 2 = 12$	$0 - 0 = 0$
High1	5	$5 - 1 = 4$	$4 - 4 = 0$	$1 - 1 = 0$
High2	9	$9 - 2 = 7$	$7 - 4 = 3$	$5 - 2 = 3$
High3	13	$13 - 3 = 10$	$10 - 4 = 6$	$9 - 3 = 6$

- Average Turnaround Time: $(14 + 4 + 7 + 10) / 4 = 35 / 4 = 8.75$ units
- Average Waiting Time: $(12 + 0 + 3 + 6) / 4 = 21 / 4 = 5.25$ units
- Average Response Time: $(0 + 0 + 3 + 6) / 4 = 9 / 4 = 2.25$ units

Simulator Results

1. Round Robin Scheduling

GANTT CHART



PID	Arrival	Burst	Priority	CT	TAT	WT	RT
LowPri	0	2	10	2	2	0	0
High1	1	4	1	10	9	5	1
High2	2	4	1	12	10	6	2
High3	3	4	1	14	11	7	3

Avg TAT: 8

Avg WT: 4.5

Avg RT: 1.5

2. Preemptive Priority Scheduling

GANTT CHART



PID	Arrival	Burst	Priority	CT	TAT	WT	RT
LowPri	0	2	10	14	14	12	0
High1	1	4	1	5	4	0	0
High2	2	4	1	9	7	3	3
High3	3	4	1	13	10	6	6

Avg TAT: 8.75

Avg WT: 5.25

Avg RT: 2.25

Analysis Questions

1. Which algorithm gave better average waiting time?

Round Robin (4.5 units compared to 5.25 units in Priority).

2. Which algorithm gave better average response time?

Round Robin (1.5 units compared to 2.25 units in Priority).

3. Did higher-priority processes gain significant advantage?

Yes, absolutely. The High1, High2, and High3 processes completely took over the CPU in the Priority algorithm, finishing much earlier collectively than they did in Round Robin.

4. Did Round Robin appear more balanced across all processes?

Yes. Round Robin interleaved all the processes smoothly. LowPri got to finish its small burst right away since it fit within one time quantum, while Priority completely left it until the end.

5. Was starvation observed or likely in Priority Scheduling?

Yes, classic starvation was strongly observed here. LowPri was preempted after just 1 unit of work and had to sit idle for 12 units of time while every single high-priority task monopolized the CPU.

6. Which algorithm would you recommend?

If fairness and overall system responsiveness are the goals, Round Robin is highly recommended as it prevents the extreme starvation LowPri experienced.

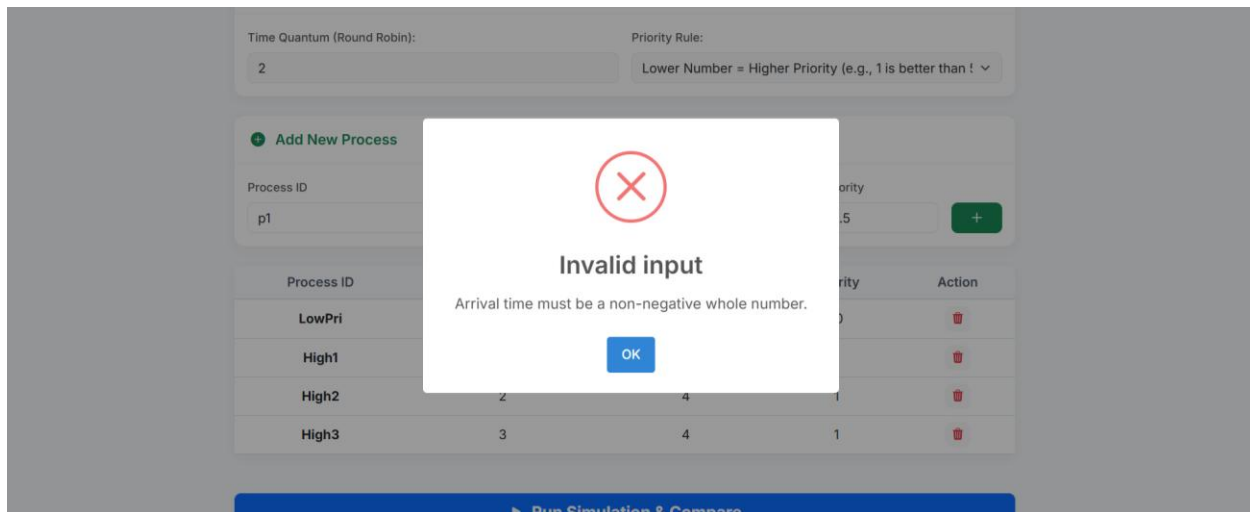
Conclusion

- **Overall Performance:** On the selected datasets, Round Robin performed better based on standard average metrics (TAT, WT, RT).
- **Urgent-Task Treatment:** Priority-based service successfully treated the three "High" processes as urgent, knocking the low-priority process completely out of the way.
- **Fairness Improvement:** Round Robin provided significantly better fairness, ensuring that the lowest priority task wasn't abandoned just because a wave of higher-priority tasks arrived.
- **Starvation Risk:** Starvation was actively observed in the Priority scheduling simulation, heavily impacting the LowPri task.

Scenario E: Validation Case

Testing how the system handle wrong input parameters thorough form validation

Process ID	Arrival Time	Burst Time	Priority
P1	-2	0	1.5



Invalid Parameters Types:

- Duplicated Process ID
- Negative arrival time
- 0 or negative burst time
- Decimal value of priority